UNIVERSITÄT
LEIPZIG

Java

Entwicklung verteilter Anwendungen

Leipzig, 14.04.2022

Dr. Pascal Kovacs

FRAGEN ZUR EINLEITUNG/MOTIVATION/ORGANISATORISCHEM

?

ZIELE

- Java als Plattform

KONTROLLFRAGEN

- Was Unterscheidet Java als Plattform von Programmen in C++?
- Welche Sicherheitskonzepte bietet Java im Vergleich zu C++?
- Welche Aufgabe hat der Garbage Collector in Java?

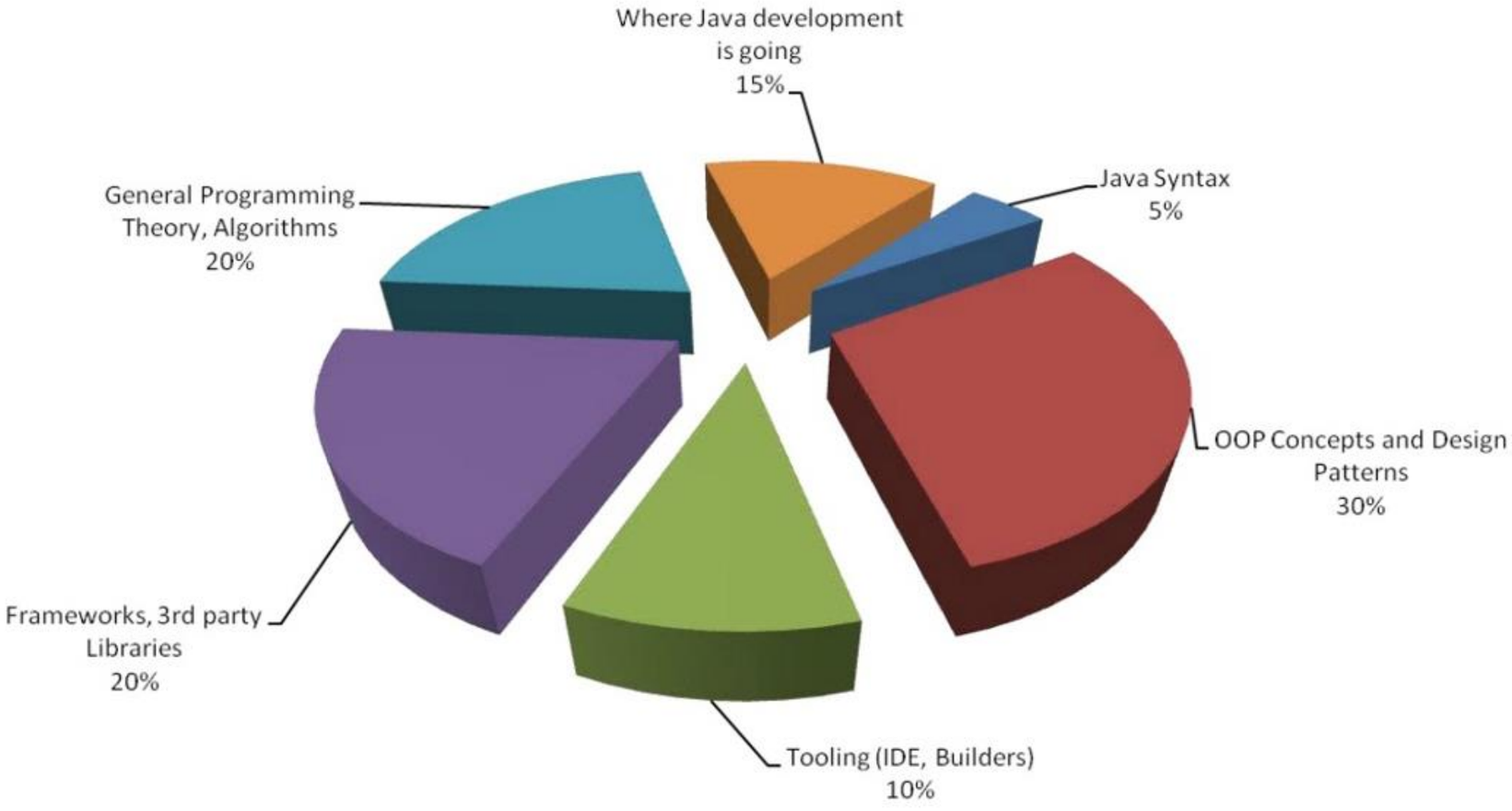
ÜBERSICHT

- Einführung in Java
- Java-Plattform

JAVA IST AUCH NUR EINE INSEL

- Java ist mehr als nur eine Programmiersprache
- Java steht auch für eine Vielzahl von Technologien
  - Java Virtual Machine als Ausführungsplattform
  - Vielzahl von Entwicklungstools wie Eclipse, JUnit
  - Applikations- und Web-Server für große, komplexe, skalierbare und sichere Geschäftssysteme
- Java besteht auch aus mehreren Communities
  - Weiterentwicklung der Sprache
  - Erstellung technischer Spezifikation
  - Implementierung von Referenzen zu Spezifikationen

LEARNING JAVA IS LEARNING…



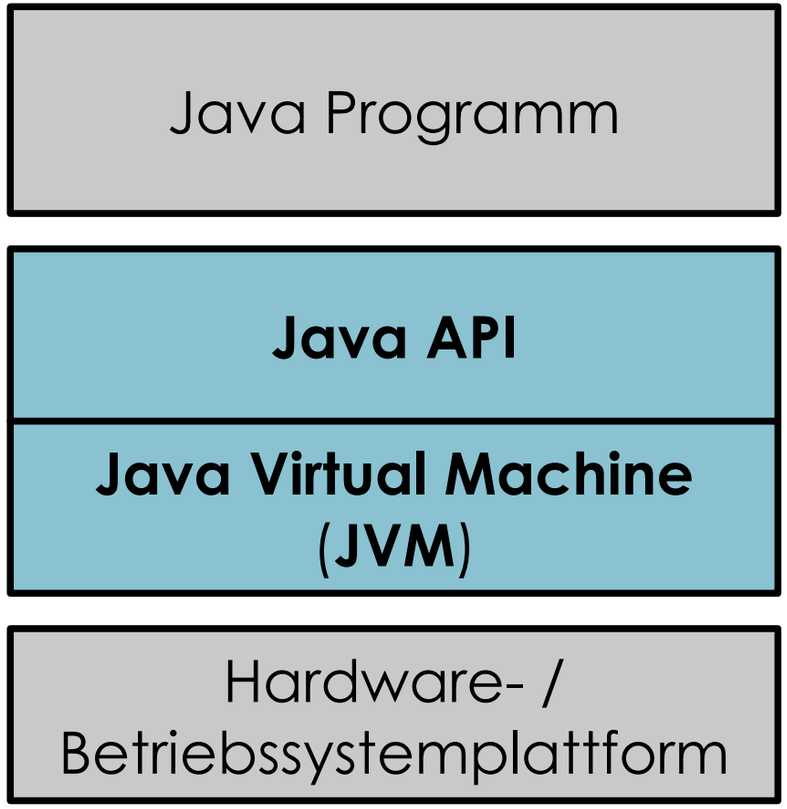
GESCHICHTE IN KÜRZE

- Entwickelt im Auftrag Sun Microsystems (1991/1992)
- 1996 erste Java-Version
- Plattformunabhängigkeit („Write once, run anywhere“)
- Ursprünglich für Haushaltgeräte, speziell Consumer-Elektronik
- Erhöhtes Maß an Sicherheit bei der Ausführung nötig
- Dann Umorientierung in Richtung Web: Applets
- Heute meistverbreitete Programmiersprache
- Deckt von einfachen Tools und wissenschaftlichen Prototypen bis zu kommerziellen Massenverarbeitungsplattformen alles ab

ÜBERSICHT

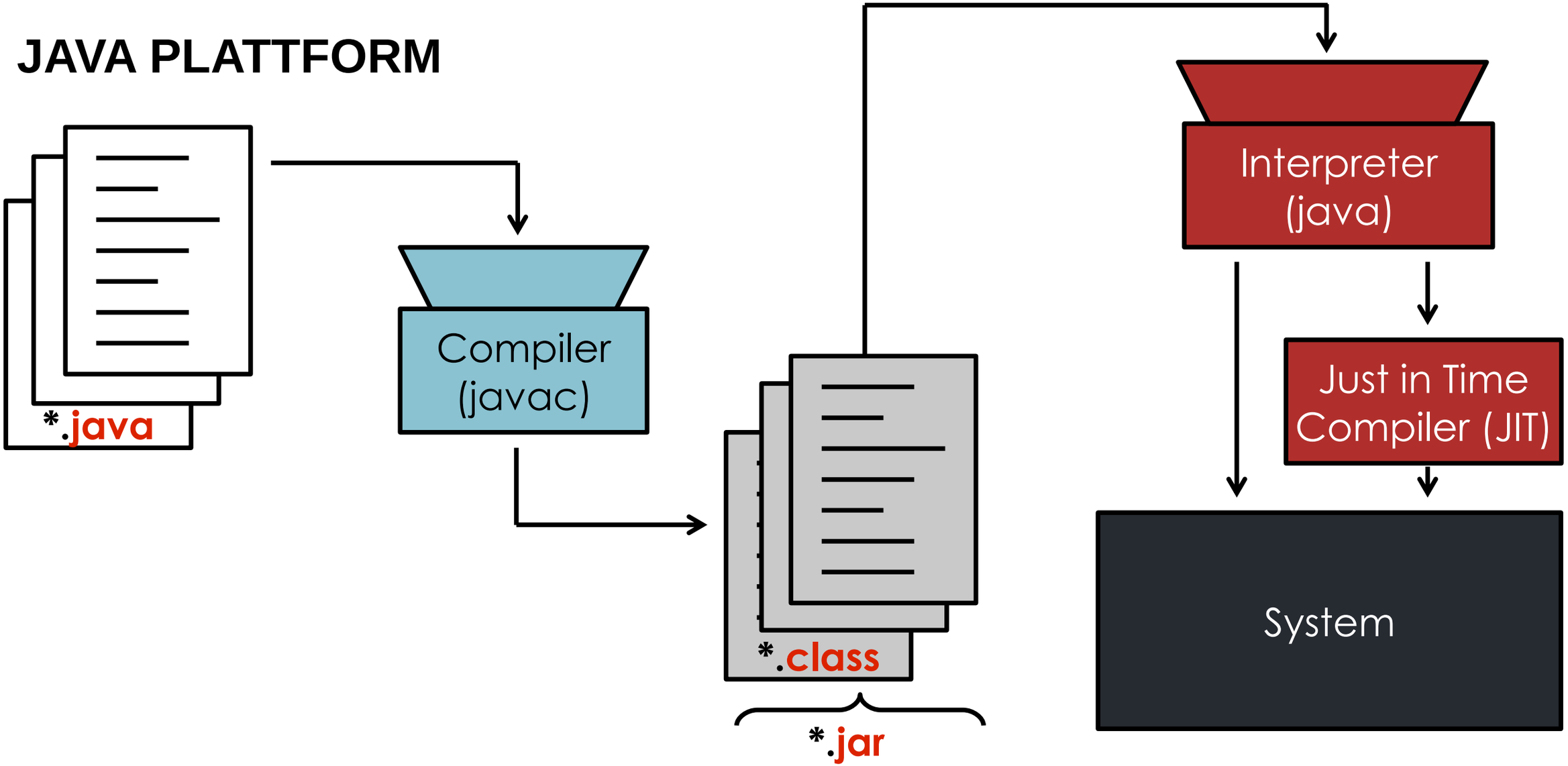
- Einführung in Java
- Java-Plattform

JAVA PLATTFORM



Java Plattform

JAVA PLATTFORM



Backup

JAVA VIRTUAL MACHINE (JVM)

- Abstraktion von dem darunter liegenden Betriebssystem
- Auf allen Systemen gleicher Befehlssatz und Zeichenkodierung
- Java-Quellcode wird zu Bytecode kompiliert
- Durch die JVM zur Laufzeit interpretiert
- Optimierung des Bytecodes durch JVM vor und während Ausführung
- Zusätzliche Übersetzung der Programmanweisungen in Maschinencode der jeweiligen Plattform durch JIT-Compiler
- Dadurch an die Architektur angepasstes Programm im Speicher welches ohne Interpretation ausgeführt wird → Performanz
- Früher waren Java-Programme deutlich langsamer als C++-Programme

SICHERHEITSKONZEPTE IN JAVA

- Java ist als möglichst „sichere“ Sprache entworfen worden
- Sprachkonstrukte
  - Keine direkte Speichermanipulation (keine Zeiger)
  - Automatische Überprüfung von Array-Grenzen
  - Automatische Initialisierung von Variablen primitiven Typs
  - Automatische Initialisierung von Objektvariablen mit NULL
- Zertifikate für Java-Archive mit Prüfung durch JVM zur Laufzeit
- Unterstützung von kryptographischen Algorithmen

GARBAGE COLLECTOR (GC)

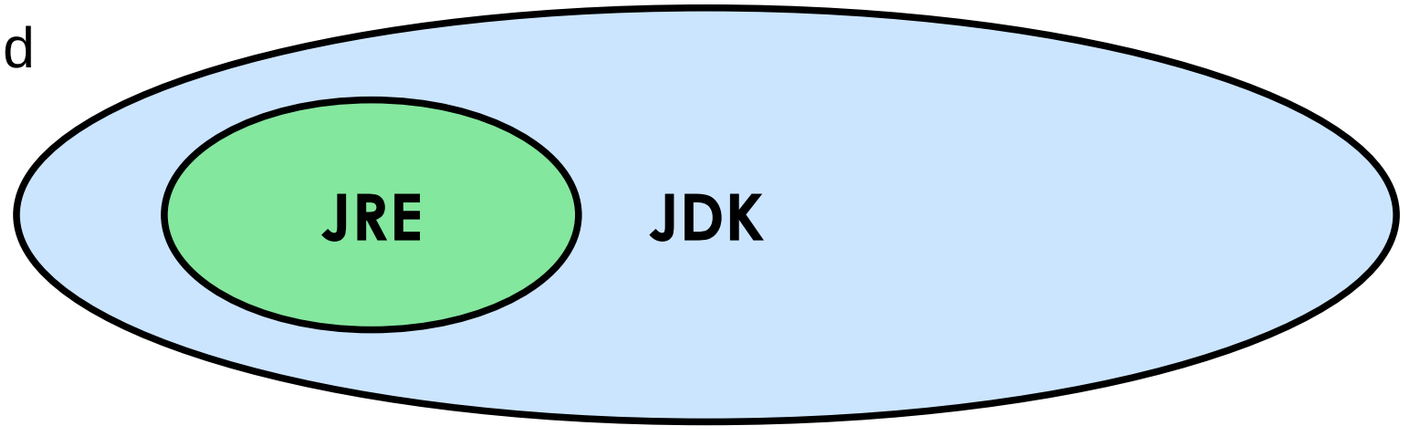
- Automatische Speicherbereinigung
- Naiver Ansatz (*Mark and Sweep*):
  - Die JVM reserviert einen bestimmten Speicherbereich (Heap)
  - Alle genutzten Objekte sind über Referenzvariablen erreichbar
  - Objektgraph wird durchlaufen und genutzte (referenzierte) Objekte werden markiert
  - Zur Markierung muss die Anwendung komplett gestoppt werden
  - Freigabe des Speicherbereichs, welcher von nicht mehr genutzten (nicht mehr referenzierten) Objekten belegt wird
  - Getrennte Speicherbereiche für Objekte kurzer (wird häufiger durch GC durchlaufen) und langer „Lebensdauer“
  - GC – Hintergrundthread niedriger Priorität

JAVA EDITIONEN

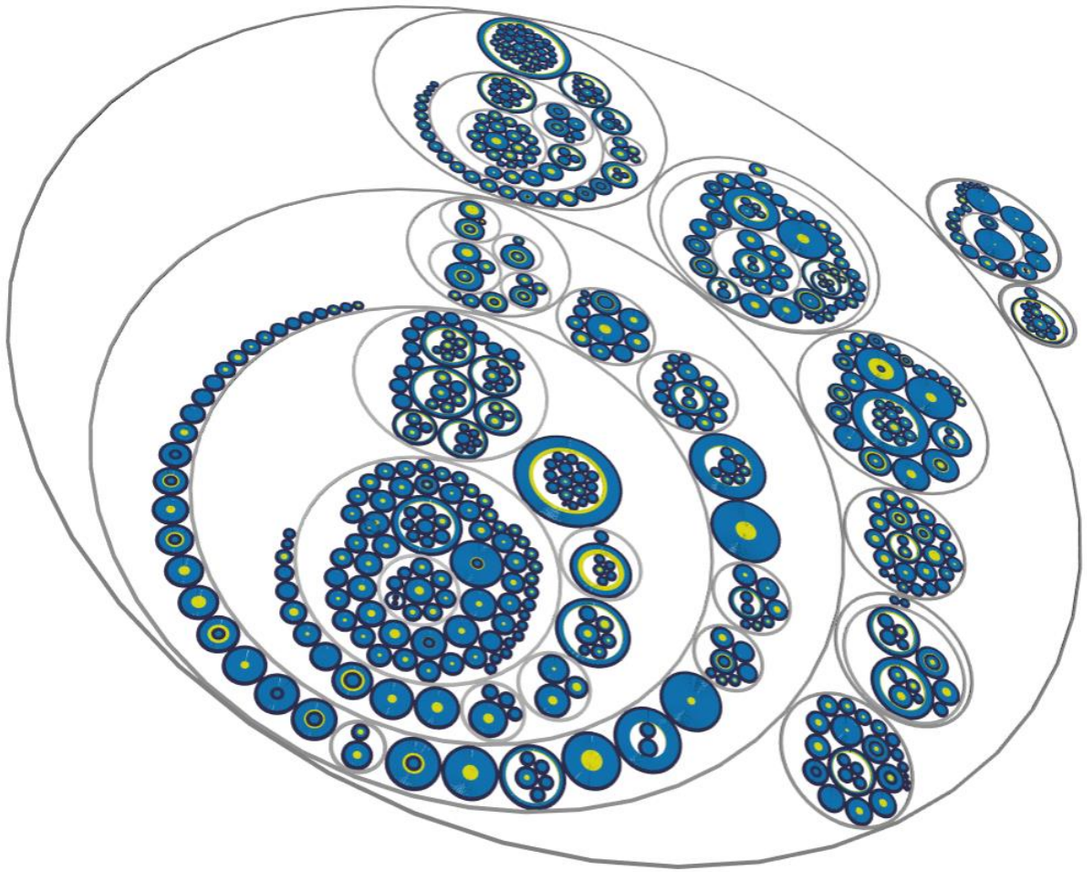
- *„One size doesn't fit it all“* (Sun Microsystems)
- Java Plattform, Standard Edition (Java SE)
  - Laufzeitumgebung zur Entwicklung, Verteilung und Ausführung sicherer und portabler Java-Programme
- Java Plattform, Enterprise Edition (Java EE)
  - Industriestandard für skalierbarer serverseitiger Java-Programme
  - Aufbauend auf der SE
- Java Micro Edition (Java ME)
  - Weit verbreitete Laufzeitumgebung für Kleingeräte wie Handys & PDAs
  - Im Vergleich zur SE deutlich abgespeckt
  - Für verschiedene Ressourcenanforderungen verschiedene Profile

JRE VS. JDK

- Java Runtime Environment (JRE)
  - Laufzeitumgebung für Java-Programme
  - Spezifisch für die jeweilige Hardware-/Betriebssystemkombination
- Java Development Kit (JDK)
  - Enthält komplette JRE
  - Zusätzlich Entwicklungs- und Überwachungswerkzeuge (z. B. Compiler, Debugger, Profiler usw.)



UNIVERSITÄT
LEIPZIG



VIELEN DANK!

Pascal Kovacs
Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de
https://www.wifa.uni-leipzig.de/iwi/